MNLP Homework 1

Sahar Khanlari **Marco Natale**

1 Introduction

This homework addresses the task of classifying cultural items based on their global, representative, or exclusive cultural significance. The objective is to design two classifiers: one based on language models and one without.

2 Non-LM Methodology

We frame the task of classifying cultural items into *cultural agnostic*, *cultural representative*, or *cultural exclusive* as a node classification problem over a knowledge graph. This approach allows us to exploit the relational structure between entities in the dataset.

2.1 Use of a Knowledge Graph

The decision to use a knowledge graph is motivated by the inherently structured nature of Wikidata. A knowledge graph enables us to represent these relationships explicitly, allowing the learning model to leverage neighborhood information and shared properties across connected entities.

2.2 Knowledge Graph Construction

The knowledge graph is constructed by querying Wikidata using the SPARQL interface. We retrieve not only the main items of interest but also their relations to other items through culturally meaningful properties. These relationships help to encode semantic proximity and cultural context.

To mitigate API rate limits, the data retrieval functions incorporate exponential backoff mechanisms. The graph is built using `NetworkX`, where each node represents a Wikidata item, and edges represent the relations.

2.3 Graph Neural Network for Classification

To classify the items, we apply a Graph Convolutional Network (GCN) using the `torch_geometric` library. The GCN model operates on the constructed graph and performs message passing to aggregate features from neighboring nodes.

The input features are derived from a combination of one-hot encodings (e.g., for categories) and text-based features using TF-IDF followed by dimensionality reduction via Truncated SVD. The model is trained to predict the cultural label of each node, optimizing a cross-entropy loss function.

2.4 Experiments

We trained a GCN composed of two hidden layers with 64 and 96 units respectively, using ReLU activations and a dropout rate of 0.3. The model was trained for 300 epochs [Figures 1 and 2] with the Adam optimizer (learning rate 0.005, weight decay 5e-4) and cross-entropy loss. Performance was monitored on a validation set.

3 LM Methodology

3.1 Model and Tokenisation

The language–model system is built with the **`distilbert-base-uncased`** transformer, loaded through the `AutoModelForSequenceClassification` and `AutoTokenizer` utilities from the Hugging Face `transformers` library. For every cultural item we concatenate all textual fields except `label` and `item` using the literal string `" [SEP] "` as a separator (cf. the `concat_fields` function). The resulting sequence is tokenised with padding/truncation to a maximum length of 512 tokens.

3.2 Training Setup

Labels are mapped to integer IDs and stored in the `labels` column. A `DataCollatorWithPadding` handles dynamic padding during batching. Training is driven by a `Trainer` that receives:

- the tokenised `train` and `validation` splits;

- a `compute_metrics` callback that reports **accuracy** and **weighted F_1** using the `evaluate` library;
- a `TrainingArguments` object whose core hyperparameters are filled in at run time from an Optuna study (see below).

After the search the model is re-initialised and trained once more with the best hyperparameters and finally saved to `"lm_model/final_model"`.

3.3 Hyperparameter Search

An automatic search is performed with the **Optuna** backend via `Trainer.hyperparameter_search`. Each trial samples from the following space (defined in `optuna_hp_space`):

- learning rate $\in [10^{-6}, 5 \times 10^{-5}]$ (log-uniform);
- weight decay $\in [0.0, 0.1]$;
- `per_device_train_batch_size` $\in \{16, 32\}$;
- `num_train_epochs` $\in \{2, 3, 4, 5\}$.

Thirty trials (`n_trials = 30`) are executed, and the objective is to *maximise* the validation weighted F_1 returned by `compute_metrics`. The hyperparameters of the trial that yields the highest F_1 are injected into a fresh `TrainingArguments` instance (`best_args`) for the final training run.

4 Results

Table 1 compares the two models on the validation set. Figures 3 and 4 show the confusion matrices for the graph-based (non-LM) and language-model classifiers, respectively.

4.1 Per-class behaviour

- **GCN.** F_1 is 0.80 for *agnostic* items but drops to 0.50 for *representative* and 0.53 for *exclusive*. The graph helps when an item links to many different neighbours, yet gives little signal when an item belongs to a single culture.
- **Language model.** F_1 is 0.91, 0.67, and 0.62 for *agnostic*, *representative*, and *exclusive* respectively. The LM beats the GCN on both minority classes, lifting F_1 by +0.17 points on *representative* and +0.09 on *exclusive*.

4.2 Why the language model scores higher

- **More text clues.** Names and descriptions often include phrases like "traditional Italian dish" or "Japanese shrine". The LM sees these words directly; the GCN only sees a short TF-IDF vector.
- **Built-in world knowledge.** DistilBERT was pre-trained on a large multilingual corpus, so it already knows many culture-related terms.

4.3 Main takeaway

The language model improves weighted F_1 from 0.63 to 0.75. Access to full text and pre-trained knowledge helps it handle the culturally specific classes that the graph model finds hard.

A Appendix

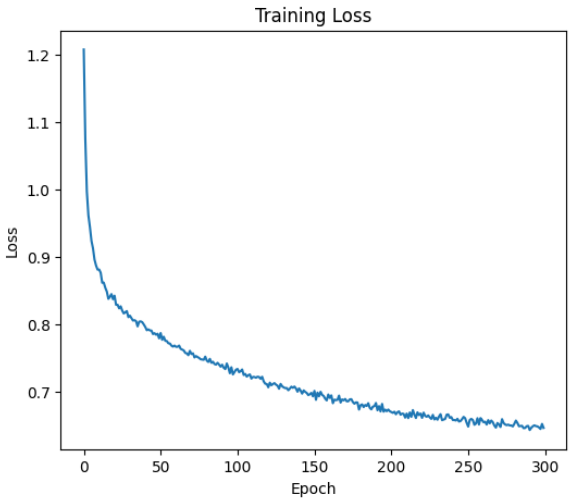

Figure 1: Training loss across 300 epochs of training of the GCN model.

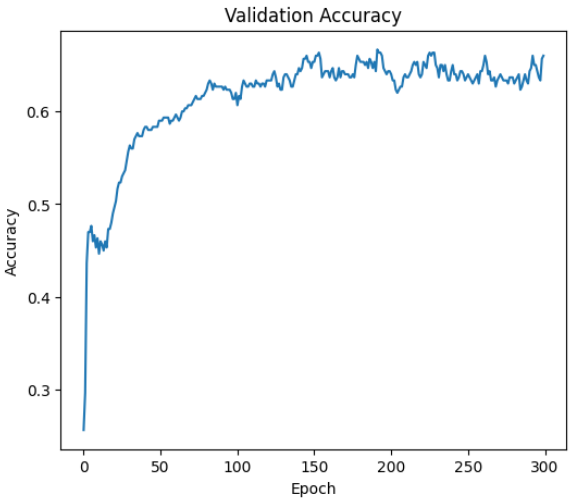

Figure 2: Validation accuracy during training of the GCN model.

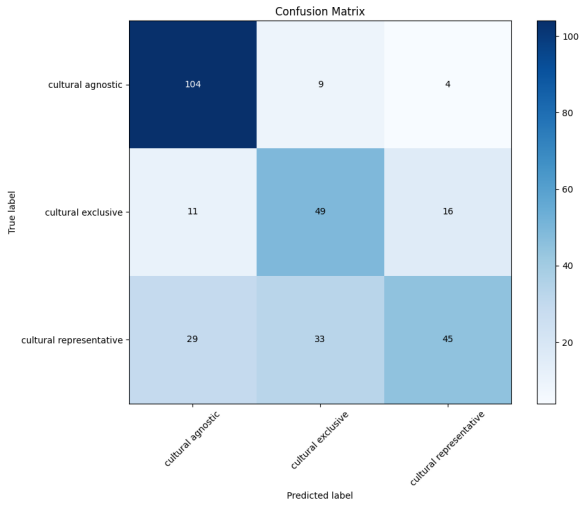

Figure 3: Confusion matrix for the GCN classifier on the validation set.

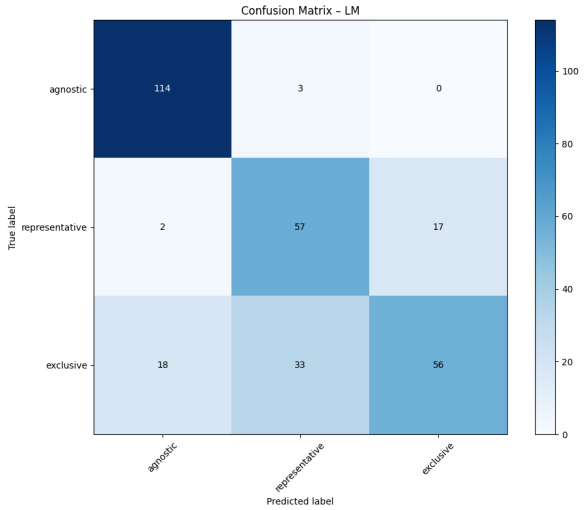

Figure 4: Confusion matrix for the LM-based classifier on the validation set.

Model	**Accuracy**	**Weighted F_1**
GCN (non-LM)	0.64	0.63
DistilBERT (LM)	0.76	0.75

Table 1: Overall validation scores.